

MULTI-STEP PROMPT WORKFLOW

Mini Project: Library Management System

Name: M. Siva Rama Krishna

Registration Number: 99230040684

Date: 07-09-2026

Scenario: Transforming a mini-project idea into a structured project proposal

1. Project Overview

Project Title: Library Management System

The Library Management System is a simple web-based application designed to help librarians manage books, student or member details, book issues, returns, and availability records. The project aims to reduce manual library work, keep records organized, and make it easier to search and maintain library information.

Suggested Technology: HTML, CSS, JavaScript, Python, Flask, MySQL

Users: Students or library members, librarians, and administrators

Core Features: Book entry, member/student entry, book search, book issue, book return, availability tracking, database storage, and basic library reports.

2. Raw Project Notes

Many educational institutions maintain library information manually in registers or separate spreadsheets. This can make it difficult to search for books, track issued and returned books, check availability, and maintain accurate member records.

The proposed Library Management System should allow librarians to add book and member details, search records, issue and return books, check book availability, save information in a database, and view simple library records. The interface should be easy for beginners to use.

STEP 1 — ANALYSIS & PROJECT PLANNING

Exact Prompt Used

Act as an expert software project analyst and technical documentation writer. I want to develop a mini project called "Library Management System". Analyze the raw project notes below and produce: (1) the main problem statement, (2) exactly 4 key project objectives, (3) the target users, (4) exactly 6 core functional requirements, (5) a simple technology stack using HTML, CSS, JavaScript, Python, Flask, and MySQL, and (6) a structured outline for a 700-word mini-project proposal. Do not invent advanced features that are not supported by the notes.

Output format: Problem Statement – Objectives – Target Users – Functional Requirements – Technology Stack – Proposal Outline

Raw notes: Many educational institutions maintain library information manually in registers or separate spreadsheets. It becomes difficult to search for books, track issues and returns, check availability, and maintain member records. The proposed Library Management System should allow librarians to add book and member details, search records, issue and return books, save information in a database, and view simple library records. The interface should be easy for beginners to use.

Intermediate AI Response — Live Execution Record

Problem Statement

Manual library record management can be time-consuming and makes it difficult to quickly search for books, track issued and returned books, check availability, and maintain accurate member information.

Objectives

- Organize book and member information in one system.
- Simplify the process of issuing and returning books.
- Make book and library records easier to search and maintain.
- Provide clear and simple information about book availability and library records.

Target Users

Students or library members, librarians, and administrators.

Functional Requirements

- Add and maintain book details.
- Add and maintain member/student details.
- Search books and library records.
- Record book issue transactions.
- Record book return transactions.
- Store records and support checking book availability.

Technology Stack

HTML and CSS for the web page structure and design; JavaScript for basic user interaction; Python with Flask for the application logic and web requests; MySQL for storing library records.

Proposal Outline

Introduction → Problem Statement → Objectives → Proposed Solution → Key Features → Technology Stack → Expected Benefits → Conclusion.

STEP 2 — PROPOSAL GENERATION

Exact Prompt Used

Excellent. Using only the project analysis, objectives, requirements, technology stack, and proposal outline from your previous response, write a complete mini-project proposal for the Library Management System. Strict constraints: maximum 700 words; use a professional academic tone; include Introduction, Problem Statement, Objectives, Proposed Solution, Key Features, Technology Stack, Expected Benefits, and Conclusion; explain how the system works in simple terms; do not introduce features or technologies not included in Step 1; use short paragraphs of no more than 3 sentences each; include this workflow: User → Web Interface → Python/Flask Application → MySQL Database → Library Records/Report. Output only the first draft of the proposal.

Intermediate AI Response — Live Execution Record

Introduction

The Library Management System is a simple web-based application developed to organize and manage common library activities. It helps librarians maintain book and member information and makes issue, return, search, and availability tracking easier.

Problem Statement

Managing library records manually through registers or spreadsheets can require considerable time and effort. It can also make it difficult to search records, track issued and returned books, check availability, and maintain accurate member details.

Objectives

The project has four main objectives: to organize book and member information, simplify book issue and return activities, make library records easier to search and maintain, and provide clear information about book availability and library records.

Proposed Solution

The proposed system provides a simple web interface through which librarians can enter book and member details, search records, and record book issues and returns. The entered information is processed by the Python and Flask application and stored in MySQL, allowing library records to be maintained in an organized way.

User → Web Interface → Python/Flask Application → MySQL Database → Library Records/Report

Key Features

The system includes book entry and member/student entry for maintaining basic records. It supports searching library records, recording book issues and returns, checking book availability, and storing library information in the database.

Technology Stack

HTML provides the structure of the web pages, while CSS is used for basic page design. JavaScript supports basic interaction on the web interface. Python and Flask handle the application logic and web requests, while MySQL stores the library records.

Expected Benefits

The system can reduce manual record-keeping work and make common library activities more organized. It can help librarians find records more easily, maintain issue and return information, and check book availability clearly.

Conclusion

The Library Management System provides a simple approach to managing essential library records through a web-based application. By combining a clear interface with Python, Flask, and MySQL, the project can make routine library record management easier and more organized.

STEP 3 — CRITIQUE & REFINEMENT

Exact Prompt Used

Act as a strict senior project reviewer. Critically evaluate the proposal you just wrote against the following checklist: 1. Is it 700 words or fewer? 2. Does it contain all required sections? 3. Are exactly 4 objectives represented? 4. Are the six core functional requirements from Step 1 accurately reflected? 5. Does it use only the approved technology stack: HTML, CSS, JavaScript, Python, Flask, and MySQL? 6. Is the workflow clear and technically consistent? 7. Are unsupported or exaggerated claims avoided? 8. Is the language suitable for an academic mini-project submission? 9. Are paragraphs limited to a maximum of 3 sentences? 10. Is the conclusion clear? First, provide a "Self-Critique" with a bullet-by-bullet evaluation of all 10 criteria. Identify any weaknesses and explain how they should be fixed. Then provide "Final Polished Mini-Project Proposal". Correct every issue you identify while preserving the useful content from the previous draft. The final proposal must remain within 700 words and satisfy all 10 criteria. Do not skip the self-critique.

Intermediate AI Response — Live Execution Record

Self-Critique

- **1. Word limit:** The proposal is comfortably below 700 words. No correction is required.
- **2. Required sections:** All eight required sections are included. No correction is required.
- **3. Exactly 4 objectives:** The Objectives section contains four distinct objectives. No additional objectives are introduced elsewhere.
- **4. Six functional requirements:** The proposal reflects book entry, member entry, search, issue, return, and availability/database storage. The final version should make these six requirements especially clear.
- **5. Approved technology stack:** Only HTML, CSS, JavaScript, Python, Flask, and MySQL are used. No unsupported technology is included.
- **6. Workflow:** The workflow is clear and consistent with the proposed technologies. The final version should retain the workflow exactly.
- **7. Unsupported claims:** The proposal avoids advanced features and exaggerated claims. Benefits are stated in realistic terms.
- **8. Academic language:** The tone is professional and appropriate for a college mini-project. Minor wording can be tightened for clarity.
- **9. Paragraph length:** Each paragraph is within the three-sentence limit. The final version will preserve this structure.
- **10. Conclusion:** The conclusion clearly summarizes the purpose and usefulness of the system. No major correction is required.

Final Polished Mini-Project Proposal

Introduction

The Library Management System is a simple web-based application designed to organize essential library activities. It helps librarians maintain book and member records while making searching, issuing, returning, and checking book availability easier.

Problem Statement

Manual library management using registers or separate spreadsheets can make routine record keeping time-consuming. It can also make it difficult to search books, track issues and returns, check availability, and maintain accurate member information.

Objectives

The project has four objectives: to organize book and member information, simplify book issue and return activities, make library records easier to search and maintain, and provide clear information about book availability and library records.

Proposed Solution

The proposed system provides a simple web interface where librarians can add book and member details, search records, and record book issues and returns. Python with Flask handles the application logic, while MySQL stores the library information.

User → Web Interface → Python/Flask Application → MySQL Database → Library Records/Report

Key Features

The system provides six core functional areas: book entry, member/student entry, record search, book issue, book return, and book availability checking with database storage. These functions support the main activities required for maintaining organized library records.

Technology Stack

HTML is used to create the structure of the web pages, and CSS is used for basic styling. JavaScript supports basic interaction, Python and Flask manage application logic and web requests, and MySQL stores the library records.

Expected Benefits

The system can reduce manual record-keeping work and provide a more organized way to manage common library activities. It can help librarians search information, maintain issue and return records, and check book availability more easily.

Conclusion

The Library Management System offers a simple and practical approach to managing essential library records through a web-based application. The project uses a beginner-friendly technology stack and focuses on the core requirements needed for organized library record management.

4. Chaining Dependency

Step 1 → Step 2 → Step 3

Step 1 transforms the raw Library Management System notes into a structured analysis containing the problem statement, exactly four objectives, target users, six functional requirements, technology stack, and proposal outline. Step 2 uses that analysis to create the first complete proposal. Step 3 uses the Step 2 proposal as its input, performs a ten-point critique, and produces a corrected final proposal.

5. Critique Gate

Step 3 is the required critique gate. The AI first evaluates the previous proposal against all ten criteria and then refines the proposal so that the final version remains within 700 words and follows the approved requirements.

6. Live Execution Evidence

Run all three prompts sequentially in the same assigned AI interface. Add screenshots showing Prompt 1 and its response, Prompt 2 and its response, and Prompt 3 with the self-critique and final response; if the platform provides a shareable chat link, include it in the submission.

7. Assessment Checklist Requirement — Evidence in this workflow

- 3+ chained prompts — Three sequential prompts are provided.
- Explicit dependency — Steps 2 and 3 explicitly use preceding outputs.
- Critique gate — Step 3 contains a 10-point review and correction stage.
- Mini-project scenario — Library Management System is used as the project.
- Live testing — Actual outputs and screenshots should be inserted after running the prompts.

8. Submission Note

Replace each “Live Execution Record” placeholder with the exact AI-generated output obtained by running the three prompts in sequence. Add screenshots or the chat link as required by the assessment instructions.